Практическая работа

Отчёт по практической работе № 4

Проектирование защищённой архитектуры приложения

Дисциплина:	Основы разработки защищённого программного обеспечения и компьютерных сетей
Вариант:	19 - Система учёта транспортных средств организации
Исходная версия:	v0.3
Версия работы:	v0.4 - Secure Architecture
Студент:	Мостовая Полина Андреевна
Группа:	ИУСбд-02-23

2026

Содержание

1 Цель работы 2

2 Исходное состояние и предметная область 2

3 Активы и выбранные угрозы 2

4 Архитектурные компоненты и границы 3

5 Архитектурная схема 3

6 Модель доступа и аутентификация 5

7 Валидация, конфигурация и аудит 5

8 Матрица угроз и защит 6

9 Структура проекта и исходный код 8

9.1 `app/security/validation.py` 8

9.2 `app/security/access_control.py` 9

9.3 `app/audit/audit_logger.py` 10

9.4 `app/services/vehicle_service.py` 11

10 Тестирование и демонстрация 13

11 Остаточные риски и ограничения 14

12 Прослеживаемость 14

13 Git и состояние версии 16

14 Ответы на контрольные вопросы 17

15 Вывод 18

1. Цель работы

Спроектировать архитектуру защиты для системы учёта транспортных средств, связать компоненты с угрозами v0.3 и реализовать небольшой проверяемый прототип: предметную валидацию, контроль ролей и объектов, сервисную операцию и аудит. Результат должен отделять действующие проверки от проектных механизмов.

2. Исходное состояние и предметная область

Проект варианта 19 предназначен для ведения реестра автопарка, назначений, поездок, пробега, неисправностей, обслуживания и отчётности. В исходном commit версии v0.3 приложение загружало и проверяло настройки `APP_NAME`, `APP_ENV`, `APP_SECRET_KEY`, затем печатало безопасное стартовое сообщение. Было три теста. Полноценной аутентификации, веб/API-интерфейса, базы данных, учёта ТС и поездок и аудита не было.

v0.4 сохраняет `app.main` и `app.config`, добавляя модули валидации, авторизации, аудита и учебные сервисные операции над in-memory данными. В проекте нет СУБД, сетевого интерфейса или реального входа пользователя. Архитектура и тесты сверены с практической работой №4, отчётами №2–3 и фактическим состоянием репозитория.

3. Активы и выбранные угрозы

К ключевым активам относятся `AS-2` профили и роли, `AS-3` конфигурация и секрет, `AS-4` реестр ТС, `AS-5` назначения и поездки, `AS-6` техническое состояние и `AS-7` аудит. Сохранены значения ID и связи из предыдущих работ.

Таблица 1: Угрозы v0.3, включённые в анализ v0.4

ID	Сценарий	Активы	Риск до мер	Статус
TH-01 / RS-01	Подбор учётных данных сотрудника	AS-1, AS-2	L=3, S=2, R=6	Мера проектируется
TH-03 / RS-03	Фиктивная выдача/смена ответственного	AS-2, AS-5	L=2, S=2, R=4	Назначения не реализованы
TH-04 / RS-04	Некорректная карточка или статус ТС	AS-4, AS-6	L=2, S=2, R=4	Частичная проверка
TH-05 / RS-05	Подмена пробега/отчёта	AS-5	L=2, S=2, R=4	Валидация реализована
TH-06 / RS-06	Повышение роли через запрос	AS-2	L=2, S=3, R=6	Авторизация прототипа проверена
TH-07 / RS-07	Отрицание изменения роли	AS-2, AS-7	L=1, S=3, R=3	Аудит частичный
TH-08 / RS-08	Чтение чужого маршрута	AS-5, AS-8	L=3, S=3, R=9	Объектный отказ проверен
TH-09 / RS-09	Утечка секрета через диагностику	AS-3	L=1, S=2, R=2	Штатный вывод безопасен

Оценки R=L×S перенесены из исходного реестра, а не пересчитаны по результатам unit-тестов. Проектное снижение баллов риска не объявляется достигнутым.

4. Архитектурные компоненты и границы

Архитектура - небольшой модульный монолит с отдельными обязанностями: CLI/вход, Validation, Authentication, Authorization, Services, Repository, Storage, Audit и Configuration. Validation, Authorization, Audit и Configuration имеют код; сервисы реализованы только для демонстрационного потока. Authentication и Repository проектируются, долговременное Storage отсутствует.

Фактические точки входа: запуск конфигурации командой `python -m app.main`; демонстрационные команды `python -m app.demo` с параметрами `--validation` и `--authorization`; переменные окружения и локальный `.env`. Веб-формы, API, загрузка файлов и подключение к СУБД не реализованы и не открыты. Параметры demo передаются вызовам Python-сервиса, а не принимаются как аутентифицированная роль от клиента.

Таблица 2: Границы доверия и контроль

ID	Поток	Контроль
TB-01	Клиент – приложение	Ввод недоверенный; проверка типа/границ данных, затем trusted principal, роль и объект. Реального HTTP-клиента нет.
TB-02	Приложение – хранилище	В будущей СУБД - доступ сервисов через repository и минимальные права; сейчас работает только память процесса.
TB-03	Приложение – аудит	Событие фиксированного формата без пароля, секрета и маршрута; ограничение доступа и ротация пока проектируются.

5. Архитектурная схема

Схема различает реализованные, частично реализованные и проектируемые группы. Входы окружения, CLI и будущие операции показаны отдельно.

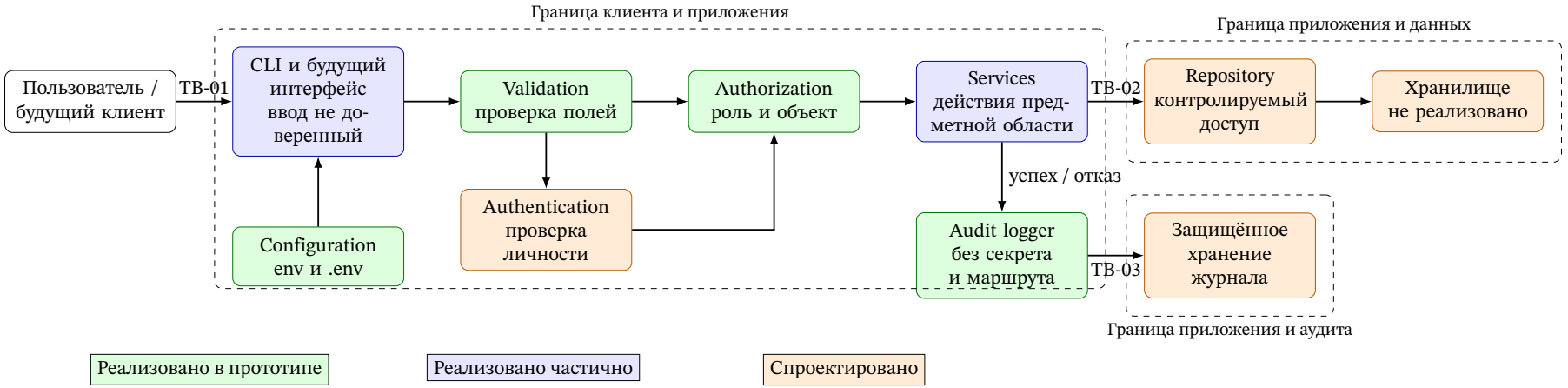


Рис. 1: Защищённая архитектура системы учёта транспортных средств v0.4.

Редактируемый исходник схемы находится в `docs/secure_architecture.mmd`, векторный экспорт – в `reports/lab4/figures/secure_architecture.pdf`.

6. Модель доступа и аутентификация

Аутентификация не реализована. Для будущего серверного интерфейса проектируются verifier пароля вместо исходного пароля, ограничение повторных попыток, выдача и отзыв сеансов, а также серверное создание доверенного контекста. Демонстрационный `Principal` - искусственный контекст теста; значение роли от клиента не может считаться доказательством личности.

Таблица 3: Сокращённая матрица разрешений

Операция	Водитель	Диспетчер	Руководитель	Администратор безопасности
Смена состояния ТС	Запрет	Разрешено	Запрет	Запрет
Чтение поездки	Только назначенная	По служебной задаче	Запрет	Запрет
Сводный отчёт	Запрет	По служебной задаче	Только агрегат	Запрет
Чтение аудита	Запрет	Запрет	Запрет	Разрешено
Управление ролями	Запрет	Запрет	Запрет	Разрешение по процедуре

Неаутентифицированный субъект не получает защищённые полномочия. Неизвестная роль и неизвестная операция запрещаются. Администратор безопасности не получает автоматическое чтение маршрутов.

7. Валидация, конфигурация и аудит

Модуль валидации проверяет единый формат ASCII-идентификаторов ТС и поездки длиной до 32 символов, допустимые статусы ТС, целочисленный неотрицательный пробег, монотонность показаний и описание дефекта не длиннее 500 символов. Верхний предел километров не придумывается. Ошибка возникает до изменения in-memory состояния. Поскольку в предыдущих требованиях нет таблицы переходов статусов, прототип проверяет допустимое конечное значение, но не вводит дополнительные правила переходов.

Конфигурация наследуется из v0.3: переменные окружения/локальный `.env`; имя и режим проверяются; `APP_SECRET_KEY` не короче 32 символов; `.env` исключён Git, пример содержит placeholder. Ошибочное значение приводит к отказу при запуске.

Аудит содержит UTC-время, субъект, действие, объект и исход. Допустимые действия и результаты ограничены списком, идентификаторы проходят форматную проверку; успехи, отказы и ошибки валидации фиксируются через `logging`. Модуль не настраивает глобальный root logger при импорте и не получает секреты или подробности поездки. Долговременная защита и политика хранения не реализованы.

8. Матрица угроз и защит

Таблица 4: Угроза - компонент - защита - остаточный риск

№ / угроза	Компонент	Механизм	Почему снижает риск	Остаточный риск
1 TH-01	Authentication	Лимит попыток и отзыв сеанса (проект)	Снижает возможность повторного подбора и срок использования чужого сеанса	Входа и лимита нет; не проверено
2 TH-03	Service, repository	Допустимые переходы и история ответственного (проект)	Запрещает недопустимые изменения и сохраняет причину/субъекта	Назначения не реализованы
3 TH-04	Validation, authorization	Проверка статуса и роль диспетчера	Неподходящее поле отклоняется до изменения; тест фиксирует отсутствие мутации	Прототип обрабатывает лишь статус одного ТС
4 TH-05	Validation, authorization	Монотонность пробега; сверка владельца поездки	Отсекает отрицательное/обратное показание и чужой объект	Нет сохранения отчёта или БД
5 TH-06	Authorization	Явный список прав, deny by default	Запрос не расширяет серверную роль; негативные роли проверены тестами	Нет реальной аутентификации
6 TH-08	Authorization, service	Сверка владельца поездки до выдачи объекта	Чужой водитель получает отказ до возврата записи	Проверено только на объекте памяти
7 TH-09	Configuration, audit	Безопасное стартовое сообщение и фиксированный формат аудита	Секрет и содержимое поездки не передаются логгеру	Инфраструктурная диагностика не проверена полностью; SEC-9/SEC-10, AT-15
8 TH-07	Audit	Субъект, действие, объект, время и исход	Запись позволяет наблюдать зафиксированный исход операции	Нет неизменяемого долговременного журнала

9. Структура проекта и исходный код

```
app/
  config.py, main.py, demo.py
  security/validation.py, access_control.py
  services/vehicle_service.py
  audit/audit_logger.py
tests/
  test_main.py, test_validation.py, test_access_control.py,
  test_audit_logger.py, test_secure_flow.py
docs/secure_architecture.md, docs/secure_architecture.mmd
reports/lab4/report.tex, report.pdf, figures/, screenshots_manual/
```

Ниже включены полные исходники трёх требуемых модулей и сервисной связки из рабочей копии.

9.1. app/security/validation.py

```
"""Проверка недоверенных данных предметной области автопарка."""

from dataclasses import dataclass
import re


class ValidationError(ValueError):
    """Входные данные не соответствуют предметным правилам."""


@dataclass(frozen=True)
class VehicleStatusChange:
    vehicle_id: str
    new_status: str


@dataclass(frozen=True)
class MileageReport:
    trip_id: str
    start_km: int
    end_km: int
    defect_description: str = ""


VEHICLE_STATUSES = frozenset({"available", "assigned", "maintenance"})
_IDENTIFIER_PATTERN = re.compile(r"[A-Za-z0-9][A-Za-z0-9._:-]*\Z")


def _required_text(value: object, field: str, maximum: int) -> str:
    if not isinstance(value, str):
        raise ValidationError(f"{field}: ожидается текст")
    normalized = value.strip()
    if not normalized:
        raise ValidationError(f"{field}: значение обязательно")
    if len(normalized) > maximum:
        raise ValidationError(f"{field}: превышена допустимая длина")
    return normalized


def _identifier(value: object, field: str) -> str:
```

```
    """Проверить ограниченный ASCIIкод- сущности, безопасный для журналирования.
    """
    normalized = _required_text(value, field, 32)
    if not _IDENTIFIER_PATTERN.fullmatch(normalized):
        raise ValidationError(f"{field}: недопустимый формат идентификатора")
    return normalized


def validate_vehicle_status_change(vehicle_id: object, new_status: object) ->
    VehicleStatusChange:
    """Проверить идентификатор ТС и допустимое новое состояние."""
    normalized_id = _identifier(vehicle_id, "vehicle_id")
    normalized_status = _required_text(new_status, "new_status", 24).lower()
    if normalized_status not in VEHICLE_STATUSES:
        raise ValidationError("new_status: неизвестное состояние ТС")
    return VehicleStatusChange(normalized_id, normalized_status)


def validate_mileage_report(
    trip_id: object, start_km: object, end_km: object, defect_description:
    object = ""
) -> MileageReport:
    """Проверить отчёт по поездке без произвольного ограничения пробега."""
    normalized_trip = _identifier(trip_id, "trip_id")
    if isinstance(start_km, bool) or not isinstance(start_km, int):
        raise ValidationError("start_km: ожидается целое число")
    if isinstance(end_km, bool) or not isinstance(end_km, int):
        raise ValidationError("end_km: ожидается целое число")
    if start_km < 0 or end_km < 0:
        raise ValidationError("Пробег не может быть отрицательным")
    if end_km < start_km:
        raise ValidationError("Конечный пробег меньше начального")
    if not isinstance(defect_description, str):
        raise ValidationError("defect_description: ожидается текст")
    description = defect_description.strip()
    if len(description) > 500:
        raise ValidationError("defect_description: превышена допустимая длина")
    return MileageReport(normalized_trip, start_km, end_km, description)


def validate_trip_id(trip_id: object) -> str:
    """Применить к запросу поездки те же правила идентификатора, что и к отчёту.
    """
    return _identifier(trip_id, "trip_id")
```

9.2. app/security/access_control.py

```
"""RBAC и проверка доступа к конкретным объектам."""

from dataclasses import dataclass
from enum import Enum
import re


class Role(str, Enum):
    ANONYMOUS = "anonymous"
    DRIVER = "driver"
    DISPATCHER = "dispatcher"
```

```
    MANAGER = "manager"
    SECURITY_ADMIN = "security_admin"


@dataclass(frozen=True)
class Principal:
    """Доверенный контекст, который должен создавать слой аутентификации."""

    subject_id: str
    role: Role


@dataclass(frozen=True)
class Trip:
    trip_id: str
    driver_id: str
    route: str


PERMISSIONS = {
    "vehicle.status.change": frozenset({Role.DISPATCHER}),
    "trip.read": frozenset({Role.DRIVER, Role.DISPATCHER}),
    "audit.read": frozenset({Role.SECURITY_ADMIN}),
    "role.change": frozenset({Role.SECURITY_ADMIN}),
}
_SUBJECT_ID_PATTERN = re.compile(r"[A-Za-z0-9][A-Za-z0-9._:-]{0,63}\Z")


class AccessDenied(PermissionError):
    """Операция запрещена матрицей доступа."""


def authorize(principal: Principal, action: str, resource: object | None = None)
    -> None:
    """Разрешить только явно перечисленные операции; неизвестные запрещены."""
    if not isinstance(action, str):
        raise AccessDenied("Операция запрещена")
    if (
        not isinstance(principal, Principal)
        or not isinstance(principal.subject_id, str)
        or not _SUBJECT_ID_PATTERN.fullmatch(principal.subject_id)
        or not isinstance(principal.role, Role)
    ):
        raise AccessDenied("Нет доверенного субъекта")
    if principal.role not in PERMISSIONS.get(action, frozenset()):
        raise AccessDenied("Операция запрещена")
    if action == "trip.read":
        if not isinstance(resource, Trip):
            raise AccessDenied("Поездка не найдена")
        if principal.role == Role.DRIVER and resource.driver_id != principal.
   subject_id:
            raise AccessDenied("Поездка не назначена этому водителю")
```

9.3. app/audit/audit_logger.py

```
"""Формирование безопасных событий аудита через стандартный logging."""

import json
```

```
import logging
import re
from datetime import datetime, timezone
from typing import Any


_ACTIONS = {"vehicle.status.change", "trip.read", "audit.read", "role.change"}
_RESULTS = {"allowed", "denied", "validation_denied"}
_IDENTIFIER_PATTERN = re.compile(r"[A-Za-z0-9][A-Za-z0-9._:-]{0,63}\Z")


def log_security_event(
    logger: logging.Logger,
    *,
    subject_id: str,
    action: str,
    object_id: str | None,
    result: str,
) -> dict[str, Any]:
    """Записать ограниченное событие без передаваемых вызывающим секретных
    полей."""
    if not isinstance(subject_id, str) or not _IDENTIFIER_PATTERN.fullmatch(
    subject_id):
        raise ValueError("Недопустимый идентификатор субъекта аудита")
    if not isinstance(action, str) or action not in _ACTIONS:
        raise ValueError("Недопустимое действие аудита")
    if not isinstance(result, str) or result not in _RESULTS:
        raise ValueError("Недопустимое действие или результат аудита")
    if object_id is not None and (
        not isinstance(object_id, str)
        or not _IDENTIFIER_PATTERN.fullmatch(object_id)
    ):
        raise ValueError("Недопустимый идентификатор объекта аудита")

    event: dict[str, Any] = {
        "timestamp": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "subject_id": subject_id,
        "action": action,
        "object_id": object_id,
        "result": result,
    }
    logger.info("%s", json.dumps(event, ensure_ascii=False, sort_keys=True))
    return event
```

9.4. app/services/vehicle_service.py

```
"""Минимальные операции учета ТС с проверками и аудитом."""

import logging
import re

from app.audit.audit_logger import log_security_event
from app.security.access_control import AccessDenied, Principal, Trip, authorize
from app.security.validation import ValidationError, validate_trip_id,
    validate_vehicle_status_change


_SAFE_SUBJECT_ID = re.compile(r"[A-Za-z0-9][A-Za-z0-9._:-]{0,63}\Z")
```

```
class VehicleService:
    """Учебный сервис над переданными in-memory данными."""

    def __init__(self, vehicles: dict[str, str], trips: dict[str, Trip], logger:
     logging.Logger):
        self.vehicles = vehicles
        self.trips = trips
        self.logger = logger

    @staticmethod
    def _audit_subject(principal: object) -> str:
        """Не обращается к недоверенному или отсутствующему principal напрямую."
    ""
        if isinstance(principal, Principal) and isinstance(principal.subject_id,
     str):
            subject_id = principal.subject_id
            if _SAFE_SUBJECT_ID.fullmatch(subject_id):
                return subject_id
        return "unknown"

    def change_vehicle_status(self, principal: Principal, vehicle_id: object,
    status: object) -> str:
        """Сменить состояние ТС только после валидации и проверки диспетчера."""
        try:
            change = validate_vehicle_status_change(vehicle_id, status)
        except ValidationError:
            log_security_event(
                self.logger, subject_id=self._audit_subject(principal), action="
    vehicle.status.change",
                object_id=None, result="validation_denied",
            )
            raise
        try:
            authorize(principal, "vehicle.status.change", change.vehicle_id)
            if change.vehicle_id not in self.vehicles:
                raise KeyError("Транспортное средство не найдено")
        except (AccessDenied, KeyError):
            log_security_event(
                self.logger, subject_id=self._audit_subject(principal), action="
    vehicle.status.change",
                object_id=change.vehicle_id, result="denied",
            )
            raise
        self.vehicles[change.vehicle_id] = change.new_status
        log_security_event(
            self.logger, subject_id=self._audit_subject(principal), action="
    vehicle.status.change",
            object_id=change.vehicle_id, result="allowed",
        )
        return change.new_status

    def read_trip(self, principal: Principal, trip_id: object) -> Trip:
        """Прочитать поездку после объектной проверки принадлежности."""
        try:
            normalized_trip_id = validate_trip_id(trip_id)
        except ValidationError:
            log_security_event(
```

```
                self.logger, subject_id=self._audit_subject(principal), action="
trip.read",
                object_id=None, result="validation_denied",
            )
            raise
        trip = self.trips.get(normalized_trip_id)
        try:
            authorize(principal, "trip.read", trip)
        except AccessDenied:
            log_security_event(
                self.logger, subject_id=self._audit_subject(principal), action="
trip.read",
                object_id=normalized_trip_id, result="denied",
            )
            raise AccessDenied("Доступ к поездке запрещён") from None
        log_security_event(
            self.logger, subject_id=self._audit_subject(principal), action="trip
.read",
            object_id=trip.trip_id, result="allowed",
        )
        return trip
```

10. Тестирование и демонстрация

Исходный набор включал три теста. Полный набор выполнен на Python 3.12.14 командой `./.venv/bin/python -m pytest -q`: результат — 45 passed, без пропусков и ошибок. Дополнительно проверены штатный запуск `app.main` и демонстрация `app.demo`. Снимки 1–4 показывают фактический запуск, весь набор тестов, валидацию, разрешённую и запрещённую операции, неизменность данных после отказа и связанные события аудита.

Таблица 5: Проверочные сценарии

Сценарий	Операция	Фактический результат	Статус
Корректное значение	Статус ТС и пробег	Валидные данные приняты	Пройдено
Некорректные данные	Пустой ID, неверный тип, отрицательный/обратный пробег	Контролируемый ValidationError	Пройдено
Административный отказ	Водитель меняет статус ТС	Отказ; статус не изменился	Пройдено
Запись отказа	Попытка водителя изменить ТС	В аудите `result=denied`	Пройдено
Объектная проверка	Чужой водитель читает поездку	AccessDenied, запись не возвращена	Пройдено
Отсутствие маршрута в логе	Владелец читает поездку	Аудит не содержит route	Пройдено

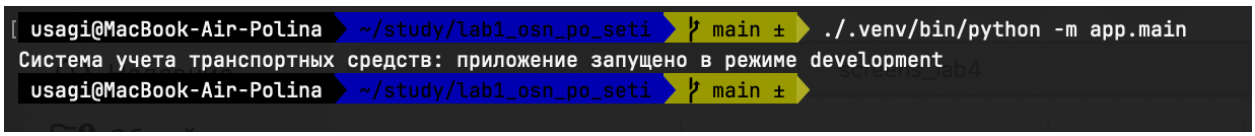


Рис. 2: Штатный запуск приложения с загрузкой конфигурации без вывода секрета.

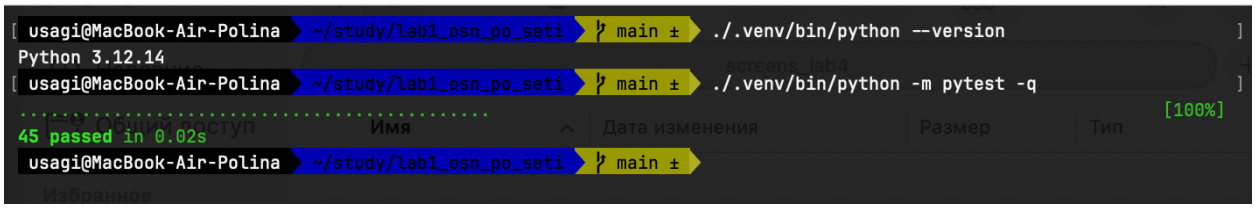


Рис. 3: Полный запуск pytest: 45 тестов пройдены.

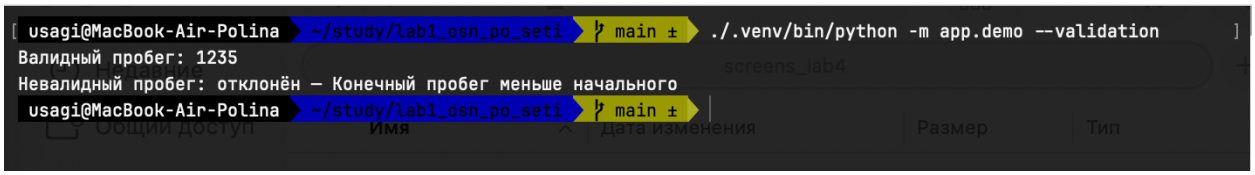


Рис. 4: Результаты проверки корректного и некорректного пробега.

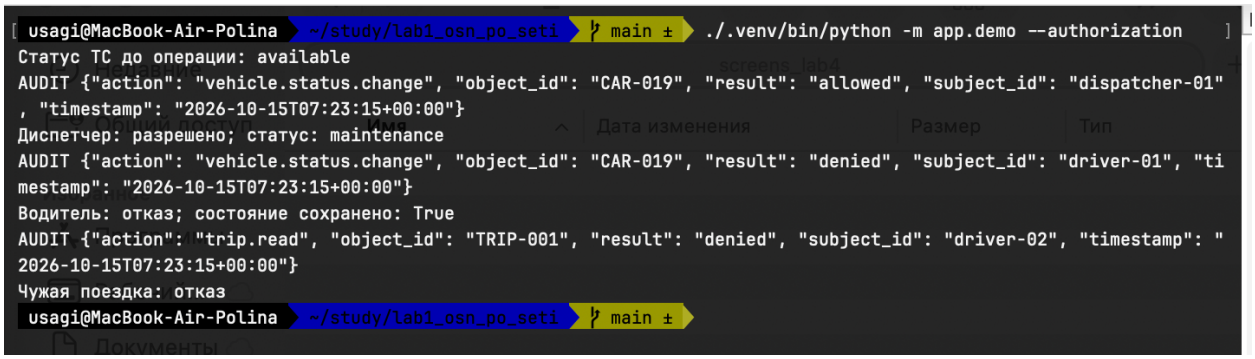


Рис. 5: Разрешение диспетчеру, отказ водителю без изменения состояния и аудит отказов.

11. Остаточные риски и ограничения

Прототип не выполняет аутентификацию; тестовые `Principal` создаются программно. Нет HTTP/API, БД, repository-слоя, сохранения поездок и пробега, управления пользователями и долговременного защищённого журнала. Локальный `logging` не обеспечивает неизменяемость, разграничение доступа, ротацию или доступность при заполнении диска. Тесты подтверждают лишь поведение указанного учебного кода. Прогнозируемое снижение RS-01/RS-06/RS-08 в v0.3 не считается подтверждённым для полной системы.

12. Прослеживаемость

Таблица 6: Связи исходных ID с прототипом

Актив / нарушитель / угроза / риск	Требование	Тест / модуль	Статус
AS-4 / N-03 / TH-04 / RS-04	SEC-4, SEC-7	`test_secure_flow`, validation/service/audit	Частично
AS-5 / N-02 / TH-05 / RS-05	SEC-6	`test_validation.py`	Валидация проверена
AS-2 / N-02 / TH-06 / RS-06	SEC-2	`test_access_control.py`	Прототип проверен
AS-5 / N-02 / TH-08 / RS-08	SEC-3	`test_access_control.py`, `test_secure_flow.py`	Объектный отказ проверен
AS-3 / N-05 / TH-09 / RS-09	SEC-9, SEC-10, AT-15	`test_main.py`, тесты аудита	Штатный запуск проверен; ошибки конфигурации покрыты не полностью
AS-7 / N-04 / TH-07 / RS-07	SEC-7	`test_secure_flow.py`	Формат события проверен

13. Git и состояние версии

Снимки 5а–5б показывают состояние до публикации: ветка `main`, HEAD на v0.3, предыдущие теги и подготовленные файлы проекта. Отчёты прошлых работ, локальные инструкции и пользовательские материалы вне области v0.4 в релиз не включаются. Финальный SHA и тег в самом отчёте не фиксируются, чтобы PDF не ссылался на commit, содержащий этот же PDF; окончательная публикация проверяется отдельно после сборки.

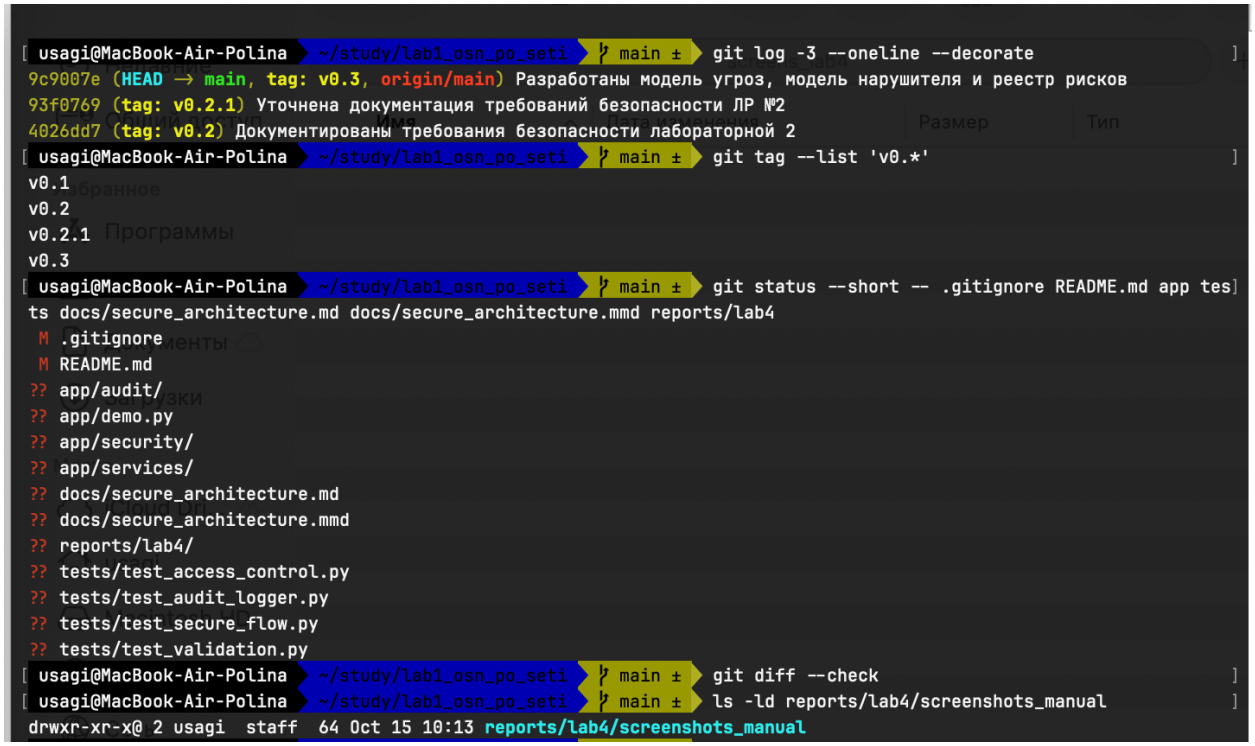


Рис. 6: История версий, подготовленные изменения и проверка diff до релиза.

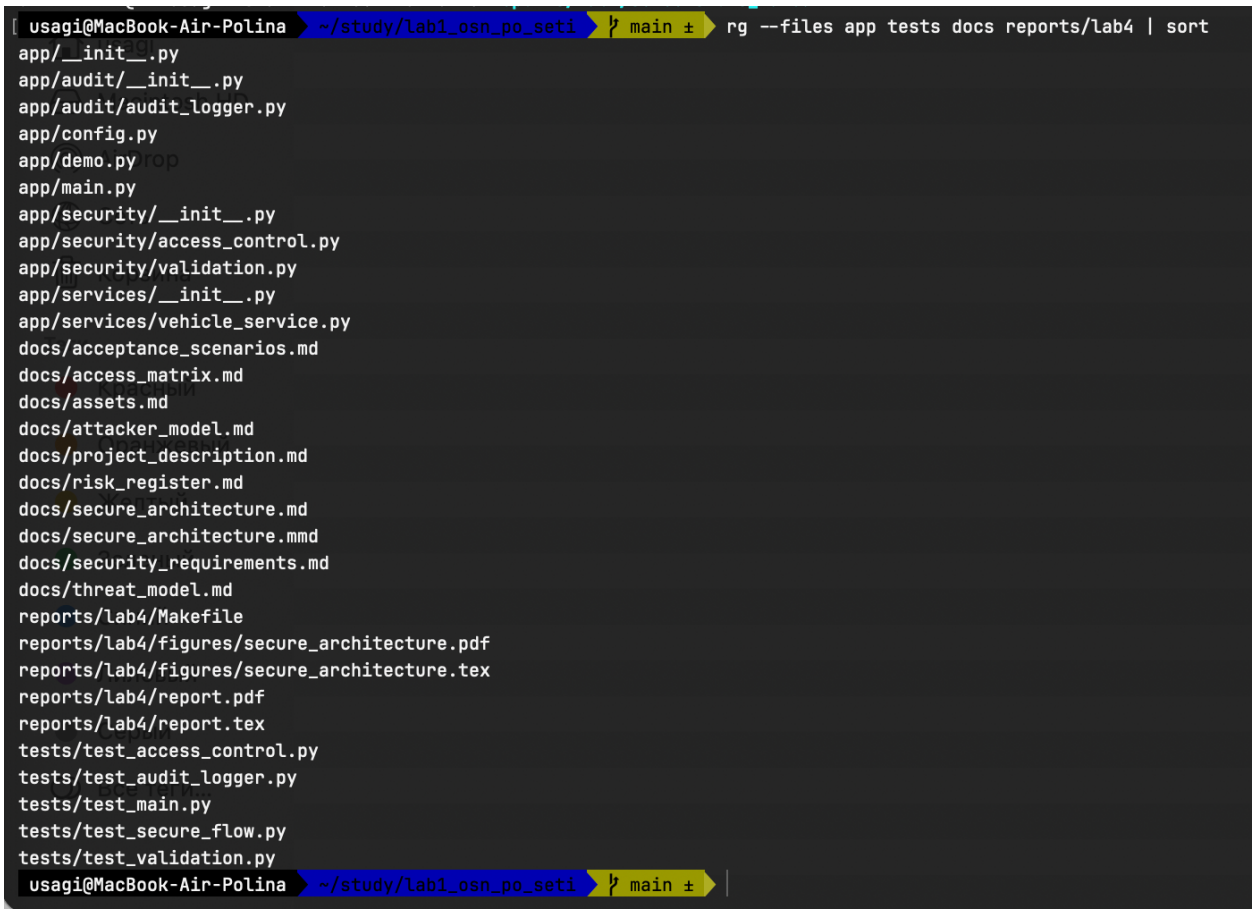


Рис. 7: Структура исходников, документации и отчёта v0.4.

14. Ответы на контрольные вопросы

1. Архитектура - структура основных частей программы и правил их взаимодействия. Здесь поток проходит от интерфейса через проверки и сервис к данным.
2. Обычная архитектура описывает устройство, защищённая дополнительно учитывает нарушителей, границы доверия, полномочия, отказ и аудит.
3. Secure by Design означает учитывать защиту при выборе структуры и интерфейсов с самого начала, например проверять право до изменения записи.
4. Defense in Depth - несколько независимых слоёв контроля: валидация, авторизация, ограниченное хранилище и аудит.
5. Минимальные привилегии дают субъекту только права, нужные его задаче; водитель не меняет состояние ТС.
6. Доверительная граница - место перехода между зонами с разным уровнем доверия, например клиент к приложению (`TB-01`).
7. Данные на границе могут быть изменены или ошибочны, поэтому тип, размер, диапазон и право на объект проверяются до операции.
8. Точка входа - интерфейс, через который данные или действия попадают в систему; пример сейчас - CLI, в будущем - веб-форма.
9. Поверхность атаки - совокупность доступных точек входа, операций, компонентов и зависимостей, которыми можно злоупотребить.
10. Разделение интерфейса, логики и хранилища не позволяет обходить правила сервиса прямой работой клиента с данными и облегчает независимую проверку.
11. Аутентификация устанавливает, кто субъект; авторизация решает, разрешено ли ему

конкретное действие над объектом.

12. Неизвестная роль не имеет доказанных полномочий; автоматическое разрешение превращает ошибку в выдачу доступа, поэтому применяется deny by default.
13. Fail Secure означает при ошибке завершить операцию безопасным отказом, не открывая доступ и не оставляя частичное изменение.
14. Журналируют вход/отказ, изменение ролей и учётных данных, операции с поездками и значимые ошибки валидации.
15. Нельзя писать пароли, токены, `APP_SECRET_KEY`, ключи, секретные поля и избыточное содержимое маршрутов.
16. Модель угроз помогает выбрать контроль для конкретного сценария до закрепления архитектуры, иначе границы и данные могут быть спроектированы без необходимой проверки.
17. Остаточный риск - риск, остающийся после выбранных мер; он не равен нулю только потому, что отдельный тест прошёл.
18. Связь показывает, какой компонент и контроль прерывает шаг угрозы, и какие условия всё ещё позволяют ей реализоваться.

15. Вывод

В v0.4 создан компактный прототип с предметной валидацией, проверкой ролей и владельца поездки, сервисом смены статуса и безопасным форматом аудита. Все 45 автоматических тестов и CLI-демонстрация подтвердили перечисленные свойства учебного стенда. Аутентификация, СУБД, сетевой API и долговременное хранилище остаются проектными. Настоящие скриншоты выполнения команд включены в разделы тестирования и Git.

1. Практическая работа №4 «Проектирование защищённой архитектуры приложения», разделы 20–28 (2026), предоставленная методичка.
2. Материалы методической работы №2: `ЛР_2.pdf`, документы проекта v0.2.
3. Материалы лабораторной работы №3: `ЛР_3.pdf`, отчёт и документы проекта v0.3.
4. Документация проекта: `docs/` и история Git до версии v0.4.
5. ГОСТ Р 56939–2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования»; ГОСТ Р 58412–2019 «Защита информации. Разработка безопасного программного обеспечения. Угрозы безопасности информации при разработке программного обеспечения».
6. NIST SP 800-218, *Secure Software Development Framework (SSDF)*; OWASP Application Security Verification Standard 5.0.0 и материалы OWASP по защищённому проектированию и моделированию угроз.
7. Шаньгин В. Ф. *Информационная безопасность компьютерных систем и сетей*; Мельников В. П., Клейменов С. А., Петраков А. М. *Информационная безопасность и защита информации*.